

Swarm Intelligence

Report on Assignment 3

Aaron Bussche

David Magaram

Jason Kasdiran

Group 28

March 9, 2026

1 Part A

1.1 Introduction

An image can be compressed by reducing the number of distinct colors in it. This process is called color quantization. The objective of finding the optimal color palette for a given image can be seen as a clustering problem. In this assignment, we tackle the color quantization problem using Particle Swarm Optimization (PSO). We implement standard PSO to iteratively optimize image color palettes and then evaluate its performance by comparing the resulting quantization against a popular clustering method, K-Means.

1.2 Preliminary Concepts

To process the image digitally, a color is represented numerically as a 3-dimensional vector corresponding to its Red, Green, and Blue (RGB) channel intensities, and a target color palette is represented as a 2D matrix of shape $(N, 3)$, where N is the number of available colors in the palette.

To quantize an image using a given palette, we implemented a custom function to evaluate every pixel in the input image array. For each pixel, it computes the squared Euclidean distance between the pixel's RGB vector and each of the N candidate color vectors in the palette. The original pixel is then replaced by the closest color in the given palette. Testing this function on the provided image and our own image using the pre-defined 4-color palette successfully reproduced the correct target quantization (Appendix 1).

To find an optimal color palette for an image we approached it as a clustering problem and used the popular K-Means clustering algorithm. In this clustering context:

- The **data** points represent the individual pixels of the original image, defined by its (R, G, B) values.
- The clusters represent groupings of pixels with similar RGB values. The **centroid** of each cluster acts as the representative color for all pixels assigned to that cluster. Therefore, the list of the N optimal cluster centroids directly forms the optimal N -color palette.

By clustering the pixels, the goal is to minimize the intra-cluster variance, which in this case equates to minimizing the Mean Squared Error (MSE) between the original image pixels and the restricted colors of the new palette.

1.3 Methods

To map the color quantization problem to PSO, each particle in the swarm represents a candidate solution, which in this case is a list of N cluster centroids. Mathematically, a particle's position is defined as $x_i = (m_{i1}, \dots, m_{iN})$, where each centroid m corresponds to a color in the palette.

We measure the quality or "fitness" of a particle by computing an evaluation function $f(x_i, Z)$ over the image dataset Z containing M pixel points. During the evaluation, each pixel z_j is assigned to the nearest centroid m . The implementation maps the input image to the color palette by finding the minimum Euclidean distance between each pixel and the candidate colors, outputting the reconstructed

image to calculate the Mean Squared Error (MSE). The PSO algorithm stores the state of the particles and the global best particle at each iteration, allowing the swarm to minimize the MSE over time.

1.4 Experimental Setup

The PSO clustering algorithm was implemented and evaluated on three test images: "peppers", "colorful", and "very small". Because the PSO algorithm can take very long to evaluate fitness on large datasets, the "colorful" image was resized from 1328×2000 to a smaller resolution of 85×128 prior to running the optimizer.

The experimental parameters were fixed for all test cases. The target color palette size was set to $N = 4$ colors. The PSO swarm consisted of 30 particles, and the algorithm was executed for a maximum of 100 iterations. To benchmark our implementation, the standard K-Means algorithm (via `sklearn.cluster.KMeans`) was applied to the same images with $K = 4$. Both algorithms were quantitatively compared based on the final MSE of the quantized image.

1.5 Results

The PSO implementation successfully quantized all three images to a 4-color palette and optimization curves were generated to show the convergence over 100 iterations. The final Mean Squared Error (MSE) values comparing PSO and K-Means are summarized below:

- **Peppers:** PSO achieved a final MSE of 524.25, slightly outperforming K-Means, which achieved an MSE of 524.26.
- **colorful:** K-Means performed better with an MSE of 1358.31, while PSO settled at a final MSE of 1373.19.
- **Very Small:** PSO significantly outperformed K-Means, reaching a final MSE of 4324.48 compared to the K-Means MSE of 4565.67.

The historical states of the global best particle showed that PSO achieved the most dramatic reductions in MSE during the first 30 to 40 iterations, after which the curves converges.

1.6 Discussion

The results confirm that Particle Swarm Optimization can successfully be applied to color quantization and is capable of finding optimal, or near-optimal, color palettes. On two of the three tested images ("peppers" and "very small"), the PSO method discovered a color palette that performed better than the K-Means baseline. This highlights the advantage of PSO as a global search algorithm; it avoids the heavy reliance on strict initial centroid placements that K-Means suffers from.

However, the experiment also illustrated some known limitations of standard PSO. The fact that K-Means performed better on the "colorful" image demonstrates that PSO can still easily get stuck in local optima during the clustering search space. Additionally, PSO is highly computationally demanding and slow to converge. Evaluating the fitness of 30 particles over 100 iterations requires exhaustively assigning every image pixel to its nearest centroid thousands of times, which justified our need to downscale images to reduce runtime. Ultimately, while PSO is an effective optimization strategy for clustering, simpler alternatives like K-Means are often much faster and sufficiently accurate for typical applications.

2 Part B

2.1 Introduction

The Boids model from Craig Reynolds is one of the most popular models of swarm intelligence and group behavior. It uses boids, which are bird like objects, that follow 3 simple rules: alignment, cohesion, and separation. These simple rules govern the interactions between individual boids, which gives rise to a more complex emergent behaviors such as flocking. This is also known as swarm intelligence. Swarm intelligence is some collective behavior that emerges when individual agents interact

with each other. This can lead to flocking, fish schools, and insect swarms. The goal of this project is to implement the Boids model and explore how the swarm behavior emerges from simple rules and how modifying the parameters affect it.

— Just some info —

Explain how the implementation satisfies all the requirements - fixed speed...

The model keeps Boid speeds fixed; agents should update their direction but move at a constant speed v which is a parameter of your model. (`const move = this.multiplyVector(this.dir, this.speed)`)

Explicitly considers what should happen at the boundary of your simulation field: The boundary is wrapped. When a boid exits from one side, it reappears on the opposite side.

uses two interaction radii: one for separation, and one for cohesion/alignment. (Ask yourself: which one should be smaller and why?): The inner radius is used for separation, while the outer radius is used for cohesion and alignment. The inner radius should be smaller than the outer radius because separation requires a closer distance to trigger the behavior, while cohesion and alignment can occur at a larger distance.

excludes the boid itself from its own neighborhood when calculating the update rules, so it does not try to separate, align or cohere to itself: The implementation excludes the boid itself from its own neighborhood when calculating the update rules. This is done by ensuring that when iterating through the neighboring boids, the current boid is not included in the calculations for separation, alignment, and cohesion. (`if( p.id == x.id ) continue`)

2.2 Methods

The JavaScript framework was used to implement the two-dimensional Boids model. The model was defined on a square grid of 400x400 pixels with wrapped dimensions. 150 boids were randomly initialized, each moving at a constant speed. The position and direction of each boid were updated at each time step based on the three rules. The simulation was ran for 300 time steps and the used parameters in this project are displayed in Table 1 below.

Boids follow three simple interaction rules: alignment, cohesion, and separation. Alignment controls how each boid moves towards the average direction of the other boids nearby. Similarly, cohesion controls how quickly each boid tries to move towards the average position of the other boids around. Separation dictates how much boids try to keep a certain distance from each other to avoid collision. Together, these rules dictate how boids interact with each other. However, other than these 3 basic behavior rules, each boid has an inner and outer radius. The outer radius is associated with cohesion and alignment, while the inner radius is linked to separation. The outer radius is the boids' visual and interaction range. It is the distance within which the boid can see other boids and interact with them, and is visualized as a gray circle surrounding the boid. The inner radius, depicted by a red circle around the boid, prevents collision and is the opposite of the interaction range. If a boid enters another boids inner radius, they are repelled away, controlled by the separation weight. Obviously, the outer radius should be larger than the inner radius so cohesion and alignment can occur.

Parameter	Value
$Radius_{inner}$	10
$Radius_{outer}$	25
Alignment weight	1
Coherence weight	1
Separation weight	1
Number of boids	150

Table 1: Parameters.

To analyze the swarm, the order and nearest-neighbor quantitative metrics were used. Order parameter is the average normalized velocity of the Boids. Order parameter tells us to what degree the swarms align in their directions, and the output is squashed between 0 and 1, 0 meaning completely random directions and 1 meaning perfect alignment. The nearest-neighbor distance is the average nearest-neighbor distance across all boids. It measures the total average distance across all boids and can tell us when individual boids get separated from the swarm or when individuals get too close to each other.

2.3 Results

Visuals. Quantitative results. Parameter experiments.

Is there swarm alignment? How does alignment change over time? How do boids maintain spacing? How do parameters influence behavior?

Time	0	100	200	300
Order parameter	0.0208	0.0753	0.105	0.616
Nearest-neighbor distance	13.956	5.925	4.278	3.405

Table 2: Quantitative results for default parameters.

Time	0	100	200	300
Order parameter	0.096	0.509	0.751	0.695
Nearest-neighbor distance	14.466	9.076	8.053	7.506

Table 3: Quantitative results for alternate parameters.

It is important to note that due to the random initializations the results can greatly vary between runs.

It is possible to reach high alignment in 300 steps with these parameters.

The visualizations in Figure 1 show the positions of the boids at different time steps. Four snapshots were taken at time steps 0, 100, 200, and 300, and are separated into an initial state, final state, and two intermediate states.

Red color represents the inner radius, while the black bar coming out of it indicates in which direction each boid is facing. The outer radius is shown as a gray circle and is each boids' interaction range.

150 boids were initialized randomly on a 400x400 field with wrapped boundaries. Boids then move around, updating their positions and orientations based on the alignment, coherence, and separation rules. Boids interact with each other by coming in each other interaction range, forming flocks as they align and cohere with each other, while also maintaining a certain distance to avoid collisions. Over time, larger flocks form as boids and smaller flocks join together. However, as shown in Figure 1d, not all boids are able to find a flock by time step 300 and remain isolated until they bump into another boid or flock.

The boids are distributed across the simulation field, and their positions and orientations change over time as they interact with each other based on the alignment, coherence, and separation rules.

Over time, we can observe the formation of flocks as the boids align and cohere with each other. The separation behavior can also be seen as boids maintain a certain distance from each other to avoid collisions.

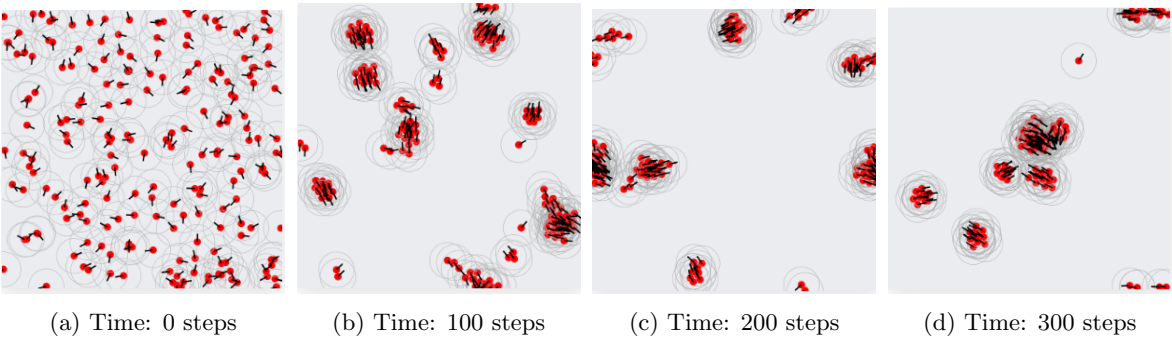
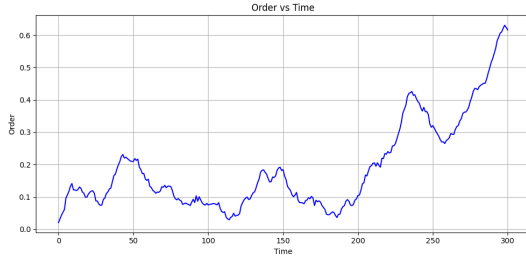
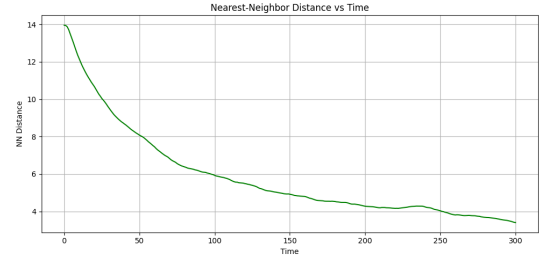


Figure 1: Visualizations of boids at four different time steps using default parameters.



(a) Order over time



(b) Nearest-neighbor distance over time

Figure 2: Plots of quantitative results over time.

The Table 3 below displays all the parameter configurations that were explored in order to investigate the effect of different parameters on the behavior of the system. These parameters were chosen because they allow for the exploration of all interesting configurations of the system.

By exploring different parameter configurations, interesting behaviors can be observed. By setting the coherence weight to 0, boids no longer strive to move towards the center of mass of their neighbors, instead, they align with their neighbors and keep a certain distance from them. This leads to an interesting behavior where boids form large nets that span the entire simulation field. Because of that, after a short while all boids start moving in the same direction. By increasing the outer radius, this can be sped up as boids are initialized with larger interaction radius and start forming large nets sooner. On the other hand, if the inner radius is increased to be more than half the value of the outer radius, this uniform behavior is disturbed. Ultimately, coherence weight plays a role in the sizes of the flocks that are formed, where higher values lead to larger flocks and lower values lead to tighter flocks.

Parameter	Value
$Radius_{inner}$	10, 40, 70, 100
$Radius_{outer}$	25, 50, 75, 100
Alignment weight	0, 0.5, 1
Coherence weight	0, 0.5, 1
Separation weight	0, 0.5, 1
Number of boids	100, 200, 300

Table 4: Explored parameter configurations.

2.4 Discussion

Why behaviors emerge. How the parameters affect the behaviors. What are the limitations of the model? How to improve it?

Be sure to come to a final conclusion and discuss some potential limitations as well.

2.5 Conclusion